TCS 704 — Advanced Computer Architecture

Detailed Study Notes — Unit 1 & Unit 2

B.Tech CSE, Semester VII

Subject Code	TCS 704	**Credits**	3
L-T-P	3-0-0	**Category**	DSC
Pre-requisite	Logic Design and Computer Organization (TCS308)		
Unit 1	Basics of Computing (9 Contact Hrs)		
Unit 2	Memory Hierarchy Design (10 Contact Hrs)		
Key Text	Kai Hwang, "Advanced Computer Architecture", 3rd Ed., McGraw Hill		
Reference	Hennessy & Patterson, "Computer Architecture: A Quantitative Approach"		

Contents

UNIT 1 — Basics of Computing

1. What is Computer Architecture?
2. Technology Trends & Moore's Law
3. Classification of Parallel Computers (Flynn's Taxonomy)
4. Performance-Based Computing & the CPU Performance Equation
5. The "Myopic View" of Computer Architecture
6. Energy, Power and Cost Trends
7. Dependability
8. Processor Speed, Cost, Power Consumption & Fabrication Yield
9. Performance Metrics, Benchmarks, Amdahl's Law & Gustafson's Law

UNIT 2 — Memory Hierarchy Design

1. Basics of Memory Hierarchy
2. Inclusion, Coherence and Locality Properties
3. Cache Memory Organization
4. Advanced Optimization of Cache Performance
5. Memory Technology and Optimization
6. Cache Coherence and Synchronization Mechanisms
7. Virtual Memory
8. Virtual Machines

UNIT 1 : Basics of Computing

1.1 What is Computer Architecture?

Computer Architecture is not just about the instruction set — it is the science of designing the structure and behaviour of a computer system so that it meets functional, performance, cost, and power goals. Modern usage splits it into three layers:

> **Instruction Set Architecture (ISA):** the interface between hardware and software — the set of opcodes, addressing modes, registers, and data types visible to a programmer/compiler.
>
> **Computer Organization (Micro-architecture):** how the ISA is actually implemented internally — pipeline depth, cache sizes, number of ALUs, branch predictor design, etc.
>
> **Hardware / Implementation:** the actual logic design, chip layout, circuit technology used to realize the organization.

Two computers can share the same ISA (e.g., x86) but have completely different organizations (an Intel Atom vs. an Intel Core i9), giving very different performance and power characteristics.

Why study it now?

For decades single-processor performance grew automatically thanks to faster clocks and deeper pipelines. That "free lunch" ended around 2004 because of the power wall, leading the industry toward multicore and parallel designs — which is exactly why this course spends units on parallel classification, memory hierarchy and multiprocessor architecture.

1.2 Technology Trends and Moore's Law

Moore's Law

> **Statement (Gordon Moore, 1965):** The number of transistors that can be economically placed on an integrated circuit doubles approximately every 18–24 months.

Moore's Law is *not* a law of physics — it is an empirical observation about manufacturing economics that became a self-fulfilling industry roadmap (semiconductor companies planned R&D to keep pace with it).

Applications / Consequences of Moore's Law

- **More transistors → more functionality** per chip: bigger caches, more cores, wider execution units.
- **Cost per transistor falls** even as raw chip cost stays roughly constant, driving down the price of computing power over time.
- Enabled the shift from single-core scalar CPUs → superscalar → multicore → many-core / GPU designs, because architects had "more transistors than they knew what to do with."
- **Its slow-down** (transistor scaling has become harder and slower below ~10 nm due to physical/quantum limits and heat dissipation) is a major reason parallel and heterogeneous computing became necessary — you can no longer just wait for a faster single core.

Broader Technology and Performance Trends

Hennessy & Patterson identify distinct eras of computer performance growth:

Era	Approx. Period	Main Performance Driver	Growth Rate
Pre-RISC	1970s – 1985	Semiconductor + architecture improvements together	~25%/yr
RISC era	1985 – 2004	Instruction-level parallelism, pipelining, caches, clock scaling	~35–52%/yr
Multicore era	2004 – ~2015	Power wall hit; performance shifted to multiple cores	~20%/yr
Post multicore / DSA	2015 – present	Domain-specific accelerators (GPUs, TPUs), specialization	Variable, workload-dependent

Underlying technologies that architects must track:

- **Integrated circuit logic technology** — transistor density improves ~35–40%/year historically; die size grows slowly.
- **DRAM (main memory) technology** — capacity grows fast, but *access latency* improves much more slowly than logic speed. This growing gap is precisely why Unit 2 (Memory Hierarchy) exists.
- **Magnetic/Flash storage** — capacity per dollar improves fastest of all, but access latency (especially disk) lags badly behind processor speed.
- **Network bandwidth** — historically has also scaled quickly, changing how distributed/parallel systems are designed.

> **Key idea to remember for exams:** The central technology trend is that *processor speed improves much faster than memory speed* — this speed gap is the single most important motivation for the entire memory-hierarchy unit that follows.

1.3 Classification of Parallel Computers (Flynn's Taxonomy)

Michael Flynn (1966) classified computer architectures based on the multiplicity of **Instruction streams** and **Data streams**.

		Instruction Stream: Single	Instruction Stream: Multiple
Data Stream	Single	SISD	MISD
	Multiple	SIMD	MIMD

(a) SISD — Single Instruction, Single Data

> One control unit fetches one instruction stream that operates on one data stream at a time.

Example: Classical von Neumann uniprocessor (traditional single-core CPU without any parallel execution). Even a pipelined single-core CPU is still logically SISD from a programmer's perspective.

(b) SIMD — Single Instruction, Multiple Data

> A single instruction is broadcast to many processing elements (PEs), each operating on its *own* data element simultaneously — "lock-step" execution.

Examples: Vector processors (Cray), array processors, GPUs (in their core execution model), and modern CPU SIMD extensions such as SSE/AVX/NEON.

> **Example:** Adding two arrays A[0..7] and B[0..7] element-wise. A SIMD unit with 8 lanes can execute the single instruction "ADD" on all 8 pairs in one step instead of looping 8 times.

(c) MISD — Multiple Instruction, Single Data

> Multiple processing units execute different instructions on the *same* data stream.

Rare in general-purpose computing; mainly seen in specialized fault-tolerant systems (e.g., space-shuttle flight control computers running the same data through different algorithms/hardware for redundancy checking) and some systolic-array pipeline configurations.

(d) MIMD — Multiple Instruction, Multiple Data

> Multiple autonomous processors simultaneously execute different instructions on different data — the most general and most common form of parallel computer today.

MIMD systems are further divided by memory organization:

- **Shared-Memory MIMD:** all processors share one global address space.
  - **UMA** (Uniform Memory Access) — every processor sees the same access latency to memory (e.g., symmetric multiprocessors, SMP).
  - **NUMA** (Non-Uniform Memory Access) — each processor has "local" memory that it can access faster than "remote" memory belonging to other processors.
- **Distributed-Memory MIMD:** each processor has its own private memory; processors communicate explicitly via message passing over an interconnection network (e.g., clusters, MPI-based systems).

> **Exam tip:** Multicore CPUs, computing clusters and most modern supercomputers are MIMD machines; GPUs are best described as SIMD (or the vendor term "SIMT" — single instruction, multiple threads, a GPU-specific refinement of SIMD).

1.4 Performance-Based Computing

"Performance" is central to computer architecture — nearly every design decision is a performance/cost/power trade-off. Two common performance measures for a user:

- **Response time (latency):** time between start and completion of a task — matters to an individual user.
- **Throughput (bandwidth):** total amount of work done per unit time — matters for a data-centre/server perspective.

```
Performance(X) = 1 / ExecutionTime(X)
"X is n times faster than Y" ⇒ Performance(X)/Performance(Y) = ExecutionTime(Y)/ExecutionTime(X) = n
```

The CPU Performance Equation

The execution time of a program can be decomposed into three factors — this is the single most important formula in the unit:

```
CPU Time = Instruction Count (IC) × Cycles Per Instruction (CPI) × Clock Cycle Time (T)
```

Equivalently, since Clock Rate = 1/T:

```
CPU Time = (IC × CPI) / Clock Rate
```

Factor	Depends primarily on
Instruction Count (IC)	Program, compiler, ISA (how compactly the algorithm compiles to instructions)
CPI (Cycles per Instruction)	ISA, micro-architecture (pipelining, hazards, cache misses add to CPI)
Clock Cycle Time	Hardware/fabrication technology, pipeline depth

Worked Example: A program has IC = 10^9 instructions, average CPI = 1.5, and clock rate = 2 GHz (T = 0.5 ns).
CPU Time = (10^9 × 1.5) / (2×10^9) = 0.75 s.
If the compiler is improved so IC drops to 0.9×10^9 with the same CPI and clock, the new CPU time = 0.675 s → a 1.11× speedup.

1.5 The "Myopic View" of Computer Architecture

A "myopic" (near-sighted) view of computer architecture is one that looks *only* at the processor/ISA in isolation — judging a design solely by raw clock speed or instruction count, while ignoring the rest of the system stack.

Why this view is misleading: Real delivered performance depends on the *entire* system: compiler quality, operating system overhead, memory hierarchy behaviour, I/O, and even the specific application/workload. A processor with a higher clock rate can easily be *slower* in practice than one with a lower clock rate but a better memory subsystem and compiler — the classic trap of "megahertz myth."

A non-myopic (holistic) approach to architecture evaluates the full hardware/software interface — this is exactly why quantitative approach textbooks insist on measuring real workloads/benchmarks (Section 1.9) rather than looking at ISA features or clock speed alone.

1.6 Energy, Power and Cost

Power

$$\text{Dynamic Power} \propto \text{Capacitive Load} \times \text{Voltage}^2 \times \text{Frequency}$$

Because power grows with the *square* of voltage, historically architects reduced voltage together with feature size to keep power in check as transistor counts rose. Once voltage scaling slowed down (it cannot go below certain physical thresholds), increasing frequency or transistor count directly increases power — this is the origin of the **"power wall"** that ended the era of ever-increasing clock speeds and pushed the industry toward multicore designs.

- **Dynamic power:** consumed while transistors are switching (charging/discharging capacitance).
- **Static (leakage) power:** consumed even when transistors are idle, due to current leakage — becomes proportionally significant at smaller process nodes.

Cost

Cost of an integrated circuit is dominated by the **die cost**, which itself depends heavily on manufacturing **yield** (Section 1.8). Other cost factors include: cost of ownership, packaging, testing, and the "learning curve" — the cost of manufacturing a chip falls over time as the fabrication process matures and yield improves.

1.7 Dependability

As systems become larger and more complex (and are used for critical applications), correct/continuous operation matters as much as speed. Dependability covers several related quantitative concepts:

Fault → may cause an **Error** (a deviation from correct state) → which may cause a **Failure** (system deviates from its specified behaviour, visible externally).

Term	Meaning
Reliability	Measure of continuous correct service, quantified as **MTTF** (Mean Time To Failure).
MTTR	Mean Time To Repair — average time taken to restore service after a failure.
MTBF	Mean Time Between Failures = MTTF + MTTR.
Availability	Fraction of time a system is usable/operational.

$$\text{Availability} = \text{MTTF} / (\text{MTTF} + \text{MTTR}) = \text{MTTF} / \text{MTBF}$$

Example: A server has MTTF = 995 hours and MTTR = 5 hours.
Availability = 995 / (995+5) = 995/1000 = 0.995 → 99.5% available.

1.8 Processor Speed, Cost, Power Consumption and Fabrication Yield

Processor Speed

Determined by clock rate and pipeline organization, but as shown by the CPU performance equation, raw clock speed alone does not determine actual performance — CPI (influenced heavily by memory stalls, covered in Unit 2) is equally important.

Fabrication Yield

> **Yield** = fraction of manufactured chips on a wafer that are fully functional (defect-free).

A simplified empirical yield model (negative-binomial / Bose-Einstein style model commonly taught):

$$\text{Die Yield} = \text{Wafer Yield} \times 1 \,/\, (1 + (\text{Defects per unit area} \times \text{Die Area}))^{N}$$

where N is a process-complexity parameter. Key relationships:

- **Larger die area → lower yield** (more area = higher chance of containing a defect) → higher cost per good die.
- **Cost per die = (Cost per wafer) / (Dies per wafer × Yield)** — this is why cutting-edge chips (very large, dense dies) are disproportionately expensive: yield drops non-linearly as die size grows.

> **Intuition:** Doubling die area doesn't just double cost — it can more than double it, because yield falls simultaneously. This is why architects sometimes prefer several small chiplets over one giant monolithic die (better aggregate yield).

Power Consumption (Recap in context of processor design)

Because a processor's power budget is capped (by cooling and battery/thermal design power limits), designers trade off clock frequency against the number of cores and per-core complexity — a direct link back to why MIMD multicore architectures (Section 1.3) became the dominant design after ~2004.

1.9 Performance Metrics, Benchmarks and Performance Laws

Problems with Simple Metrics

- **MIPS** (Million Instructions Per Second) — misleading because instruction sets differ in complexity; more instructions ≠ more work done.
- **MFLOPS** (Million Floating Point Ops/sec) — useful only for floating-point-heavy scientific code, not general-purpose workloads.

> The only fully reliable way to compare machines is to time *real programs / representative workloads* — this motivates benchmark suites.

Benchmark Standards

Benchmark Suite	Purpose
SPEC (CPU, INT/FP)	Industry-standard CPU benchmark suite from real applications (integer & floating-point programs) — most widely cited in research/marketing.
TPC (TPC-C, TPC-H, etc.)	Transaction Processing Performance Council — benchmarks for database/OLTP throughput.
Whetstone / Dhrystone	Older synthetic benchmarks for floating-point and integer performance respectively.
Kernels	Small, heavily-used code fragments (e.g., matrix multiply, FFT) extracted from real programs.

Benchmarks are generally combined using the **geometric mean** of normalized ratios (relative to a reference machine) rather than an arithmetic mean, to fairly combine performance ratios across very different programs.

Amdahl's Law

> Amdahl's Law gives the theoretical maximum speed-up of a task when only *part* of it can be improved/parallelized — the *unimproved* fraction eventually dominates and limits total speedup, no matter how much you improve the rest.

$$\text{Speedup} = 1 \,/\, [\ (1 - F) + F/S\ ]$$

where F = fraction of execution time that benefits from the enhancement, S = speedup of that enhanced fraction alone.

> **Worked Example:** Suppose 80% (F = 0.8) of a program can be parallelized, and we run that part on enough processors to get a 10× speedup on that portion (S = 10).
> Speedup = 1 / [(1−0.8) + 0.8/10] = 1 / [0.2 + 0.08] = 1/0.28 ≈ 3.57×.
> Note that even with infinite processors (S → ∞), the maximum possible speedup is 1/(1−F) = 1/0.2 = 5× — the serial 20% caps everything.

> **Key exam takeaway:** Amdahl's Law shows that focusing purely on making the parallel portion faster gives diminishing returns; the sequential (non-parallelizable) fraction is the true bottleneck. This is a strong argument for why reducing the serial portion of an algorithm matters more than adding more processors, beyond a point.

Gustafson's Law (listed in the syllabus table)

Note: the official syllabus lists this topic as "*Lhadma's Law*" next to Amdahl's Law; this is the standard companion result taught alongside Amdahl's Law in this exact position in every computer-architecture course, and is universally known as **Gustafson's Law** (also called Gustafson–Barsis's Law). It is presented below under its correct, standard name.

Gustafson's Law takes the opposite viewpoint from Amdahl: instead of fixing the problem size and adding processors, it assumes that as more processors become available, the *problem size scales up* too (users solve bigger problems in the same time budget), which avoids Amdahl's pessimistic ceiling.

$$\text{Scaled Speedup} = (1 - F) + F \times N$$

where N = number of processors, F = parallel fraction (measured on the parallel/scaled system).

Worked Example: With F = 0.8 (80% parallel fraction) and N = 10 processors:
Scaled Speedup = (1−0.8) + 0.8×10 = 0.2 + 8 = 8.2× — much more optimistic than Amdahl's fixed-size prediction, because here the workload itself is allowed to grow with the number of processors.

Aspect	Amdahl's Law	Gustafson's Law
Problem size	Fixed	Scales with processor count
View	Pessimistic — bounded speedup	Optimistic — near-linear speedup possible
Best suited for	Fixed workloads (real-time tasks)	Large-scale scientific/data-parallel computing

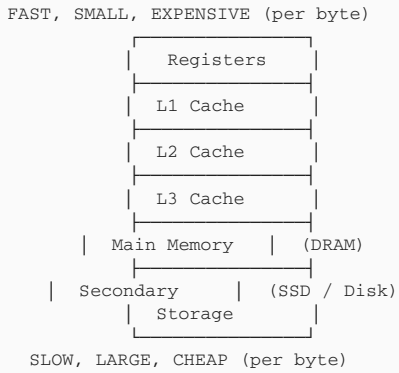
Measuring Performance — Summary Principles

- Always benchmark with representative, real workloads, not synthetic microbenchmarks alone.
- Use geometric mean to aggregate normalized performance ratios across a benchmark suite.
- Report both response time and throughput as they can favour different designs.
- Use Amdahl's Law to reason about upper bounds on speedup from any single optimization — a recurring exam-style numerical question.

UNIT 2 : Memory Hierarchy Design

2.1 Basics of Memory Hierarchy

No single memory technology is simultaneously fast, large, cheap, and low-power. The **memory hierarchy** solves this by organizing memory into multiple levels, exploiting the fact that programs don't access all their data uniformly.



> **Goal of the hierarchy:** Give the illusion of memory that is as fast as the fastest level and as large/cheap as the largest level — achieved by keeping frequently used data in the faster, smaller levels close to the CPU.

Each level acts as a cache for the level below it. The design succeeds because of the **locality of reference** exhibited by real programs (Section 2.2).

Key Hierarchy Parameters

Parameter	Meaning
Hit	Requested data found in the current level.
Miss	Requested data not found; must be fetched from a lower (slower) level.
Hit time	Time to access data already present in this level.
Miss penalty	Extra time needed to fetch data from the next lower level on a miss.
Hit ratio / Miss ratio	Fraction of accesses that are hits / misses.

```
Average Memory Access Time (AMAT) = Hit Time + Miss Rate × Miss Penalty
```

> **Worked Example:** L1 hit time = 1 cycle, miss rate = 5%, miss penalty = 20 cycles.
> AMAT = 1 + 0.05×20 = 1 + 1 = 2 cycles.

2.2 Inclusion, Coherence and Locality Properties

Kai Hwang's treatment of memory hierarchy defines three fundamental properties that justify why a multi-level hierarchy works correctly and efficiently:

(a) Inclusion Property

> Data present in a smaller/faster (inner) level of the hierarchy must also be present in every larger/slower (outer) level. i.e., Cache ⊂ Main Memory ⊂ Disk.

This guarantees that if data is not found in cache, it can always be located at some outer level — no data is ever "only" in the fast cache and nowhere else (except in write-back scenarios before write-back to memory completes, a special controlled case).

(b) Coherence Property

> All copies of the same data item, across different levels of the hierarchy (or across different caches in a multiprocessor), must eventually be kept consistent — i.e., updates made in one level must be properly reflected (propagated) to other levels/copies.

Two basic update strategies at the single-processor cache-to-memory level:

- **Write-through:** every write to cache is immediately also written to main memory → memory is always consistent, but generates more traffic.
- **Write-back:** writes update only the cache; the modified ("dirty") block is written to memory later (e.g., when evicted) → less traffic but requires careful bookkeeping (dirty bits).

In multiprocessors, this same idea extends to keeping multiple *processor caches* consistent with each other — covered in depth in Section 2.6 (Cache Coherence Protocols).

(c) Locality of Reference Property

> Programs tend to access a relatively small portion of their address space at any given time — this predictability is what makes caching effective at all.

- **Temporal Locality:** if a memory location is accessed, it is likely to be accessed again soon (e.g., loop variables, frequently called functions).
- **Spatial Locality:** if a memory location is accessed, nearby locations are likely to be accessed soon after (e.g., sequential array traversal, instruction fetch of sequential code).

> **Example:** In the loop `for(i=0;i<n;i++) sum += A[i];`, accessing A[i] exhibits *spatial* locality (next element A[i+1] is adjacent in memory) and the variable `sum`/`i` exhibit *temporal* locality (reused every iteration).

2.3 Cache Memory Organization

A cache is divided into fixed-size **blocks/lines**. The mapping between a main-memory block and where it can be placed in the cache defines the organization type.

(a) Direct-Mapped Cache

> Each memory block maps to exactly **one** specific cache line, usually via: Cache Line = (Block Address) mod (Number of Cache Lines).

Pros: Simple, fast, cheap hardware (no comparison needed beyond one tag check). **Cons:** High conflict-miss rate — two frequently used blocks mapping to the same line will keep evicting each other ("thrashing").

(b) Fully Associative Cache

> A memory block may be placed in **any** cache line. Requires comparing the tag against all lines simultaneously (parallel comparator hardware).

Pros: Minimum conflict misses. **Cons:** Expensive/complex hardware (comparators for every line), and usually slower/more power-hungry — practical only for small caches (e.g., TLBs).

(c) Set-Associative Cache

> A middle ground: the cache is divided into sets; each memory block maps to exactly one set (like direct mapping) but can occupy *any* line within that set (like full association). An "n-way set-associative" cache has n lines per set.

```
2-way Set Associative Cache example (4 sets, 2 ways each):

                Set 0: [Line A] [Line B]
                Set 1: [Line A] [Line B]
                Set 2: [Line A] [Line B]
                Set 3: [Line A] [Line B]

        Set index = (Block Address) mod (Number of Sets)
```

Most real processors use set-associative caches (commonly 4-way, 8-way) — a practical compromise between hit rate and hardware cost/speed.

Cache Write Policies

Policy	Description
Write-through	Write to cache and memory simultaneously (simpler, more traffic).
Write-back	Write only to cache; mark block "dirty"; write to memory only on eviction (faster, needs dirty bit).
Write-allocate	On a write-miss, the block is first loaded into cache, then written.
No-write-allocate (write-around)	On a write-miss, write goes directly to memory, bypassing cache.

Cache Replacement Policies

Needed whenever a set/cache is full and a new block must be brought in (relevant for associative and set-associative caches):

- **LRU (Least Recently Used):** evict the block unused for the longest time — exploits temporal locality, best hit rate typically, but costlier to implement exactly.
- **FIFO (First-In-First-Out):** evict the oldest-loaded block regardless of usage.
- **Random:** evict a randomly chosen line — surprisingly effective and very cheap to implement in hardware.

Types of Cache Misses ("The 3 C's")

Miss Type	Cause

Compulsory (Cold)	First-ever access to a block — unavoidable regardless of cache size.
Capacity	Cache cannot hold all blocks needed during program execution, even with full associativity.
Conflict	Multiple blocks compete for the same set/line (occurs in direct-mapped/set-associative, not fully associative).

2.4 Advanced Optimization of Cache Performance

Techniques (largely from Hennessy & Patterson) to reduce AMAT by attacking hit time, miss rate, or miss penalty individually:

Technique	What it targets	How it helps
Larger block size	Miss rate	Exploits spatial locality; but too large increases miss penalty and may waste bandwidth.
Larger cache size	Miss rate	Reduces capacity misses; increases hit time and cost.
Higher associativity	Miss rate	Reduces conflict misses; increases hit time slightly.
Multilevel caches (L1/L2/L3)	Miss penalty	A fast small L1 backed by larger slower L2/L3 avoids going all the way to main memory on every L1 miss.
Priority to read misses over writes	Miss penalty	Reads are usually on the critical path (needed for further computation); writes can be buffered.
Avoiding address translation during cache indexing	Hit time	Use virtually-indexed, physically-tagged caches to overlap TLB lookup with cache access.
Way prediction	Hit time	Predict which "way" of a set-associative cache holds the data to avoid checking all ways.
Trace caches	Hit time / IPC	Store decoded dynamic instruction traces (used in some superscalar CPUs) to avoid re-decoding.
Non-blocking (lockup-free) caches	Miss penalty	Allow the cache to service further hits/misses while an earlier miss is still being fetched ("hit under miss").
Hardware prefetching	Miss rate	Hardware predicts and fetches blocks before they are explicitly requested, based on observed access streams.
Compiler-controlled prefetching	Miss rate	Compiler inserts explicit prefetch instructions ahead of predicted use.
Compiler optimizations (loop interchange, blocking/tiling)	Miss rate	Reorders memory accesses in source code / loop nests to better match cache-line and cache-capacity boundaries, maximizing spatial and temporal locality.

Loop Interchange Example: Accessing a 2-D array in row-major storage as `for j: for i: A[i][j]` jumps across rows (poor spatial locality). Interchanging to `for i: for j: A[i][j]` accesses memory sequentially — dramatically improves cache hit rate with zero change to program semantics.

2.5 Memory Technology and Optimization

Technology	Characteristics
SRAM (Static RAM)	Used for caches/registers. Fast (no refresh needed), but low density and expensive — 6 transistors per bit typically.
DRAM (Dynamic RAM)	Used for main memory. Higher density, cheaper (1 transistor + 1 capacitor per bit) but slower, and needs periodic refresh (capacitor leaks charge).
Flash memory	Non-volatile; used in SSDs; slower to write than DRAM, has limited write endurance (wear-levelling required), much faster than magnetic disk.
Magnetic Disk	Very high capacity per dollar, non-volatile, but mechanical latency (seek + rotation) makes it orders of magnitude slower than DRAM.

DRAM Optimizations

- **SDRAM (Synchronous DRAM):** synchronized to the system clock, allowing pipelined/burst access.
- **DDR SDRAM (Double Data Rate):** transfers data on both rising and falling clock edges, doubling bandwidth without raising the clock frequency.
- **Memory banking/interleaving:** splitting memory into multiple banks that can be accessed concurrently/overlapped, increasing effective bandwidth.
- **Wider buses / burst mode:** transfers multiple consecutive words per request, amortizing the fixed row-access overhead.

2.6 Cache Coherence and Synchronization Mechanisms

The Cache Coherence Problem

In a multiprocessor system, each processor typically has its own private cache. If two processors cache the same memory block and one

> processor modifies its copy, other processors' cached copies become stale unless a protocol keeps them consistent — this is the **cache coherence problem**.

Formal coherence requirements

- A read by processor P to location X, following a write by P to X (with no other writes in between), must return the value written by P.
- A read by a processor to X, after a write to X by another processor, must return the written value (after sufficient time and no other writes in between).
- Writes to the same location by different processors must be seen in the **same order** by all processors (write serialization).

Coherence Protocol Approaches

Approach	Description
Snooping protocols	All caches monitor ("snoop on") a shared bus; every cache watches every other cache's transactions and reacts (invalidate/update) accordingly. Works well for small-scale, bus-based systems.
Directory-based protocols	A central/distributed "directory" tracks which caches hold which blocks; coherence messages are sent only to caches known to hold a copy — scales better to large multiprocessors where broadcasting is impractical.

Common Snooping Protocol States — MSI / MESI

State	Meaning
M — Modified	Block is modified in this cache only; memory copy is stale; this cache must supply data to others.
S — Shared	Block is unmodified and may exist in multiple caches simultaneously (all consistent with memory).
I — Invalid	Block is not valid in this cache (either never loaded, or invalidated by another processor's write).
E — Exclusive (MESI only)	Block is present only in this cache, unmodified, and matches memory — allows silent transition to Modified without a bus transaction, an optimization over plain MSI.

On a write by any processor, other caches holding that block must transition to Invalid (invalidation-based protocols) — this enforces the coherence guarantees listed above.

Synchronization Mechanisms

Multiprocessors also need hardware primitives to coordinate concurrent access to shared data structures safely:

- **Test-and-Set:** an atomic instruction that reads a memory location and sets it to 1 in a single indivisible step — classic building block for locks.
- **Compare-and-Swap (CAS):** atomically compares a memory value to an expected value and, if equal, swaps in a new value — foundation for lock-free data structures.
- **Load-Linked / Store-Conditional (LL/SC):** a pair of instructions used together to implement atomic read-modify-write without a single monolithic atomic instruction.
- **Locks / Mutexes and Semaphores:** higher-level software constructs built on the above hardware primitives to enforce mutual exclusion and coordinate producer-consumer style access.
- **Barriers:** synchronization points where all participating threads/processors must arrive before any may proceed further — used to synchronize phases of parallel algorithms.

2.7 Virtual Memory

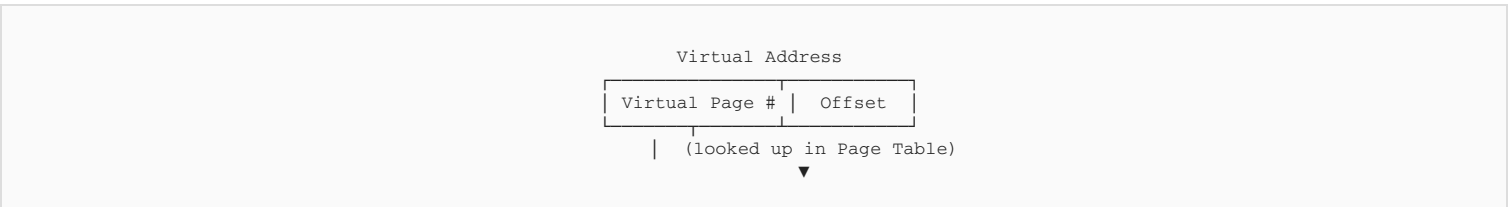
> Virtual memory is a memory-management technique that gives each process the illusion of a large, private, contiguous address space, independent of the actual amount of physical DRAM installed, by transparently mapping virtual addresses to physical addresses (and to disk when necessary).

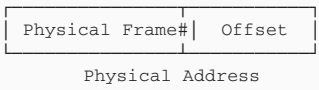
Why Virtual Memory?

- **Isolation/Protection:** each process has its own address space, preventing one process from corrupting another's memory.
- **Simplifies programming:** programmers don't need to manage physical memory placement or worry about how much RAM is actually installed.
- **Efficient sharing:** allows controlled sharing of physical pages (e.g., shared libraries) between processes.
- **Extends usable memory:** allows programs whose total size exceeds physical RAM to run, using disk as an overflow ("swap space").

Paging

> The virtual address space is divided into fixed-size **pages**; physical memory is divided into equal-sized **frames**. A **page table** (per process) maps virtual page numbers to physical frame numbers.





Page Faults

> A **page fault** occurs when a process accesses a virtual page that is not currently loaded in physical memory — the OS must fetch it from disk, possibly evicting another page first (using a page-replacement policy, e.g., LRU), then update the page table and resume the instruction.

Because a page fault involves a disk access, its penalty is enormous — often measured in *milliseconds*, compared to a cache miss penalty measured in *nanoseconds*. This huge asymmetry is why page-replacement algorithms are chosen very carefully (favouring accuracy over speed, unlike cache replacement where speed matters more).

Translation Lookaside Buffer (TLB)

> Since every memory access would otherwise require an extra page-table lookup, a small, fast, fully-associative cache called the **TLB** stores recently used virtual-to-physical page translations, avoiding repeated page-table walks (a direct application of the locality principle from Section 2.2).

TLB miss: requires a "page table walk" to find the mapping (in hardware or software depending on architecture), which is then cached in the TLB for future use.

Segmentation (brief contrast)

An alternative/complementary scheme where the address space is divided into variable-sized logical **segments** (e.g., code, stack, heap) rather than fixed-size pages — offers a more logical view matching program structure but is harder to manage efficiently than paging (leads to external fragmentation); many systems use paged segmentation, combining both ideas.

2.8 Virtual Machines

> A Virtual Machine (VM) is software that creates an abstraction of underlying hardware (or an OS/application environment) so that multiple isolated "virtual" computers can run on one physical machine.

Types of Virtual Machines

Type	Description	Example
System VM (Hardware-level)	Provides a complete virtual hardware environment on which a full guest OS can run; managed by a **hypervisor / Virtual Machine Monitor (VMM)**.	VMware, VirtualBox, Xen, KVM, Hyper-V
Process VM (Application-level)	Provides a platform-independent execution environment for a single program/process, abstracting away the underlying OS/ISA.	Java Virtual Machine (JVM), .NET CLR

Hypervisor / VMM Types

- **Type 1 (Bare-metal / Native):** runs directly on physical hardware, with no host OS underneath — used in data centres/servers for efficiency (e.g., Xen, VMware ESXi).
- **Type 2 (Hosted):** runs as an application on top of a conventional host operating system (e.g., VirtualBox or VMware Workstation running on Windows/Linux).

Requirements for an Architecture to be Efficiently Virtualizable

(Popek and Goldberg's classical virtualization requirements, foundational to this topic):

- **Equivalence:** a program running under the VMM should behave identically to running directly on real hardware (except for timing/resource availability).
- **Resource control:** the VMM must have complete control over virtualized resources — guest software must not be able to bypass it.
- **Efficiency:** a statistically dominant fraction of guest instructions should execute directly on the real hardware without VMM intervention, for acceptable performance.

Instructions that could break isolation (privileged/sensitive instructions) must trap into the VMM so it can emulate them safely — architectures with a clean separation of privileged/non-privileged instructions are much easier to virtualize efficiently (this is why hardware virtualization extensions like Intel VT-x / AMD-V were introduced, adding a new privilege level specifically for the hypervisor).

Benefits of Virtualization

- Higher hardware utilization — multiple isolated workloads share one physical machine.
- Strong fault/security isolation between VMs.
- Easier system management: live migration, snapshots, rapid provisioning of new "machines".
- Legacy software/OS support on modern hardware.

Unit 2 — Summary Table

Concept	One-line takeaway
Memory Hierarchy	Multiple levels trade speed vs. size vs. cost; works because of locality.
Inclusion/Coherence/Locality	The three properties that make a hierarchy both correct and effective.
Cache Organization	Direct-mapped (simple), fully-associative (flexible), set-associative (practical compromise).
Cache Optimization	Attack hit time, miss rate, or miss penalty — never all at once with one trick.
Memory Technology	SRAM (fast/cache) vs DRAM (dense/main memory) vs Flash/Disk (capacity/non-volatile).
Cache Coherence	Needed in multiprocessors; MESI/snooping (small scale) vs directory-based (large scale).
Virtual Memory	Gives illusion of large private memory via paging + page tables + TLB.
Virtual Machines	Abstract hardware/OS to run multiple isolated systems on shared physical hardware.

Prepared study notes based on TCS 704 syllabus (Graphic Era University) — reference texts: Kai Hwang, "Advanced Computer Architecture" (3rd Ed.); Hennessy & Patterson, "Computer Architecture: A Quantitative Approach".